

Rapport final

Studio generatif interactif - Application Flask de creativite numerique

Element	Information
Date de verification	26/06/2026
Periode de travail	04/03/2026 au 26/06/2026
Depot GitHub	https://github.com/HoSSamOuammi/Digital_Thinking
Objet du livrable	Code complet, application Flask, README et rapport final 2-3 pages.

1. Concept et direction artistique

Le projet est un studio generatif interactif qui regroupe des ateliers de generation visuelle, visualisation de donnees, traitement media et galerie. L'utilisateur transforme des parametres, des fichiers CSV ou des images en productions exportables.

La direction artistique reste sobre: interface claire, composants administratifs, couleurs limitees et textes francais. Le rendu privilegie la lisibilite et la demonstration technique.

2. Modules implements

Module	Fonction	Resultat
Tableau de bord	Page d'accueil avec resume du studio, compteurs et acces rapides.	Point d'entree clair pour presenter le projet.
Atelier generatif	Generation de visuels a partir de series, palettes, graine, densite et accents dessines.	Images exportees dans la galerie.
Donnees visuelles	Lecture CSV ou donnees de demonstration, nettoyage numerique et transformation graphique.	Visualisations en image.
Outils medias	Traitement image: noir et blanc, sepia, contours, glitch, rotation, palette dominante. Audio optionnel.	Fichiers transformes et telechargeables.
Galerie	Listing separe des images et audios generes avec pagination.	Consultation et telechargement des resultats.
Equipe	Profils, roles, emails et photos chargees depuis static/Admins.	Lien entre interface et repartition du travail.

3. Outils utilises et pipeline technique

Outil	Utilisation
Flask / Jinja2	Routes serveur, templates HTML et formulaires.
Pillow	Effets et exports d'images.
Pandas / NumPy	Lecture, nettoyage et preparation des donnees CSV.
Matplotlib	Production des visualisations de donnees.
PyDub / ffmpeg	Traitement audio lorsque l'environnement le permet.
unittest	Verification automatique des routes et parcours principaux.
Git / GitHub	Historique, collaboration et depot final public.

Pipeline de fonctionnement:

- L'utilisateur choisit une page atelier depuis le tableau de bord.
- Flask recoit le formulaire et valide les donnees utiles.
- Le module Python specialise genere ou transforme le contenu.
- Le fichier final est sauvegarde dans static/generated.
- La page affiche le resultat et propose le telechargement.
- Les tests verifient les routes, la securite CSRF, la galerie et le nettoyage des fichiers.

4. Challenges et solutions

Challenge	Solution
Code initial trop centralise	Separation en create_app, routes, formulaires, stockage, securite et modules metier.
Interface a rendre presentable en contexte scolaire	Design sobre, navigation simple, libelles francais et mise en page responsive.
Dependance audio sensible a l'environnement	Detection de disponibilite et degradation propre si ffmpeg n'est pas installe.
Fichiers generes et imports temporaires	Nettoyage automatique, pagination et limites de cache pour garder le depot propre.

5. Equipe et livrables verifies

Photo	Membre	Role
	Aya EL Amrani	Architecture Flask, configuration, formulaires, stockage et routes.
	Khadija Baskar	Traduction francaise, libelles, contenus visibles et coherence UI.
	Hossam OUammi	Integration Flask, design administratif, medias, galerie et presentation.
	Abderrahmane El Garti	Tests fonctionnels, documentation, rapport et analyse technique.

Livrable	Verification
Base de code complete	Dossiers app.py, studio, modules, templates, static, tests et requirements.txt.
Application Flask	Lancement par python app.py, routes principales testees.
README	Installation, lancement, structure, tests et notes techniques.
Rapport final	PDF de 2-3 pages avec concept, modules, pipeline, challenges et solutions.